

the basic deployment unit, we are talking about scheduling pods across nodes—starting and stopping them — depending on the scale or even a requirement to keep a certain number of pods running. A replication controller is what makes this possible, and it controls the lifecycle of the pods.

Service: This is the magical piece in the whole puzzle. You might have a single pod or a collection of them forming your applications. To achieve the goal of being able to keep the infrastructure independent from the application, you have to expose your pods or, in other words, allow outside applications to access it. This is what a service is for. It provides a single interface to access your collection of pods, making it possible for pods to be created and brought down in a seamless fashion, without affecting the interface or needing to bring the application down.

The whole thing is woven together via the concept of a label, which is a collection of key-value pairs that can identify all the above artefacts.

The discussion is incomplete without covering the Kubernetes master, which is the server that manages all the Kubernetes nodes. The master has a service discovery mechanism, a scheduler and an API server to deal with client tools (kubectl) that can manage the Kubernetes cluster.

There are several other concepts in Kubernetes like volumes, secrets, etc. I suggest that you look up the official documentation.

Container orchestration: Other players

Kubernetes definitely has taken the lead when it comes to container orchestration solutions, especially when it comes to the range of features that it supports. However, we have Docker, which has competition in the form of Docker Swarm. Docker Swarm is the native clustering solution that has been built into the Docker runtime itself. It was announced a few months back at *Dockercon* and should see traction as it builds on the features that Kubernetes already has. In addition to these two solutions, you also have Apache Mesos.

It seems likely that Kubernetes and Docker Swarm might eventually battle it out in the orchestration solutions space. Irrespective of which emerges as the frontrunner, interoperability and simplicity, which have been the key to widespread Docker usage, should be kept intact.

Use Minikube to get familiar with Kubernetes

It may seem that to run Kubernetes, one needs a cluster of machines; but this is not the case. There is a great tool called Minikube that allows you run a single-node Kubernetes cluster on your laptop.

To install it on your machine, please follow the guide available at <https://github.com/kubernetes/minikube>

Once you install it successfully, you will have the Minikube utility that neatly packages the Docker tools inside it.

To get started, we need to launch our single node cluster. To do that, give the *Start* command as shown below:

```
$ minikube start
Starting local Kubernetes cluster...
```

Kubectl is now configured to use the cluster.

At this point, you can check the status of the node via the following command:

```
$ kubectl get nodes
NAME          STATUS    AGE
minikubevm   Ready     1m
```

We can now run a default echo service that is provided by Minikube via the following command:

```
$ kubectl run hello-minikube --image=gcr.io/google_
containers/echoserver:1.4 --port=8080
```

```
deployment "hello-minikube" created
```

It will take a while to schedule the container on a pod. You can get the list of pods in your cluster via the command shown below. Note that you might have to run this command a few times, in case the pod is still starting.

```
$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
hello-minikube-2433534028-2j49e   1/1      Running   0           42s
```

It also creates a deployment that you can view as given below:

```
$ kubectl get deployments
NAME          DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
hello-minikube 1          1          1             1           1m
```

The next step now is to expose our deployment as a service. This will make it available to the outside world via a single interface. To do that, use the command given below:

```
$ kubectl expose deployment hello-minikube --type=NodePort
service "hello-minikube" exposed
```

We can check the list of services exposed in the cluster now via the command given below:

```
$ kubectl get services
NAME          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
hello-minikube 10.0.0.50    <nodes>       8080/TCP    30s
```

To get the URL with which we can access this service, use the following command:

```
$ minikube service hello-minikube --url
http://192.168.99.100:31499
```